Fortik Agent Kill Switch — MVP Technical Implementation Plan

Oct 6, 2026 · @Mitthan Meena

Summary

The MVP proves one claim on real systems: **Fortik contains a risky agent action without breaking the workflow, and can undo what already happened.** It does this by owning the agent's access: agents get short-lived, narrow credentials from Fortik, and every side-effecting call is classified green, yellow or red before it runs.

Positioning: Fortik is the authority broker for autonomous agents. Every risky action gets a short-lived credential, a policy decision, a containment level and an undo plan before it touches the business. Kill is one feature of the product, not the whole product.

In scope

- Proxy layer: agentgateway for MCP, A2A and HTTP tool traffic; Fortik's own app gateway for Gmail, Drive, GitHub and AWS; a Postgres wire-protocol proxy; a Claude Code hook
- Authority broker that issues per-action, short-lived access for Google Workspace (Gmail, Drive), GitHub, AWS and Postgres
- A visibility model that covers 20+ agent types, with a live communication map; the MVP proves 8 representative types: a LangGraph custom agent, Claude Code, a SaaS copilot connector, a finance agent, a text-to-SQL agent, a memory-heavy assistant, an orchestrator swarm and a shadow script
- Green / yellow / red classifier: deterministic rules (OPA/Rego) plus a simple per-agent volume baseline
- Control ladder (Full → Supervised → Read-only → Isolated → Quarantined → Killed) with automatic step-down and manual step-up
- Containment actions: suspend, isolate, remove memory access, and time-limited app suspensions (e.g. GitHub for 30 minutes) with automatic restore
- Swarm detection and one-click kill-swarm
- Copy count and caps per agent; per-app access narrowing
- Undo for 6 action types (listed in the Undo section)
- Agent Diary: append-only, hash-chained event log with export

- Console: the three v2 screens (Command Center, Agents, Communication Map), wired to real data

Out of scope for the MVP: ML-based detection, prompt-injection scanning, endpoint agents, multi-tenant SaaS, SSO/SCIM, Fleet E-Stop beyond a single tenant. In scope: SaaS copilots only where their enterprise access flows through Fortik-managed connectors. Out of scope: pausing or killing the vendor's own hosted runtime.

Done means: both demo scenarios (attack and accident) run end to end against real Google Workspace, GitHub, AWS and Postgres test accounts; Fortik decision overhead under 50 ms p95 on green actions, excluding the downstream API's own latency; 3 design partners running in observe mode.

Why not existing tools

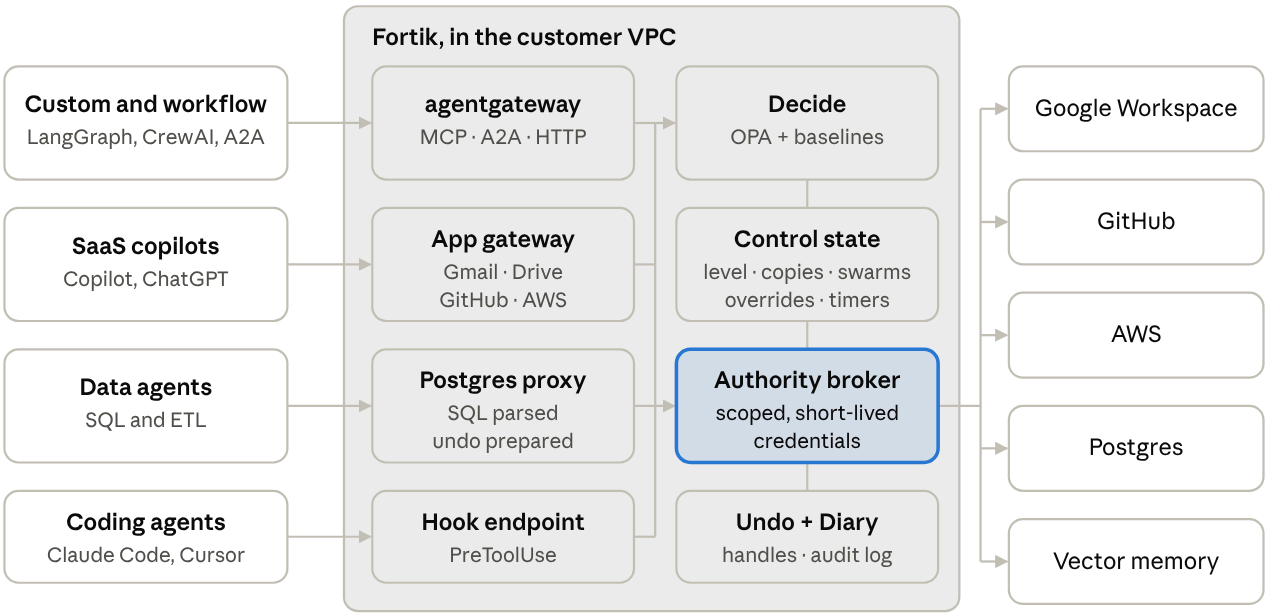
Most tools act after they see risk; Fortik issues authority per action, and only when policy allows it. Competitor descriptions below are from public materials and should be checked in demos before customer use.

Vendor class	What they do	How Fortik differs
Agent security (e.g. Straiker)	Discover agents, test them, detect and block at runtime; kill switch revokes tools, freezes memory, suspends sessions	Fortik owns the credential path: per-action short-lived credentials, a step-down ladder, and undo handles prepared before risky writes
SASE / SWG	Control network and web traffic	Fortik controls business verbs (SEND, SPEND, DELETE, GRANT) and the agent's authority, not packets
IAM and non-human identity	Grant identities standing access	Fortik grants authority per action and narrows it live; it uses IAM as the identity source
API and MCP gateways	Route, rate-limit and block calls	Fortik steps agents down, narrows access, restores it on a timer, undoes actions and records the Diary

Vendor class	What they do	How Fortik differs
EDR	Watches processes on the endpoint	Fortik controls agent actions across SaaS, databases, memory and queues, wherever the agent runs

Architecture

Four interception points, one decision, one broker



MVP architecture · 4 interception points, 5 systems

Each kind of agent traffic enters through the interception point that can read it: agentgateway for MCP, A2A and HTTP tool calls; Fortik's app gateway for Gmail, Drive, GitHub and AWS (including SaaS copilot connectors); the Postgres proxy for SQL; the hook endpoint for coding agents. All four call the same Decide service, and only the authority broker can hand out the credential that makes an action run. The console (the v2 screens) reads control state, the map and the Diary, and writes levels, overrides, swarm actions and undo through the same APIs. Proxy choices and the fallback are in Integration points.

Request path for one call: agent calls through its interception point → the adapter builds the Action → Decide reads control state (level, overrides, swarm membership, copy count) and returns class + decision → green: broker issues a credential and the call is forwarded → yellow: reduced form (narrowed scope, delay, approval, or undo handle first) → red: held, agent stepped down → every step written to the Diary and pushed to the map.

Integration points

The MVP sees agent actions in three ways and enforces them in one: **an agent cannot act on a system without a credential Fortik issued.** Interception tells Fortik what the agent wants; the broker makes the answer stick.

Proxy layer

There is no single proxy. Each kind of traffic gets the interception point that can see what matters in it, and all four call the same Fortik decide service.

Traffic	Interception point	Build or adopt	What it sees	Fail mode
Agent → tool, agent → agent (MCP, A2A, HTTP tool calls)	**agentgateway** (Rust, Apache 2.0, Linux Foundation project) as the data plane, calling Fortik decide per request	Adopt	Tool name and arguments, A2A calls between agents, delegation chain, OpenTelemetry traces for the map	Fail closed
Agent → business apps (Gmail, Drive, GitHub, AWS)	**Fortik app gateway** (Go) + authority broker	Build (core IP)	Request body: recipients, share type, repo, branch, IAM action, cost estimate; prepares undo before forwarding	Fail closed
Agent → Postgres	**Fortik Postgres proxy** (Go, wire protocol via pgproto3, SQL parsed with pg_query)	Build	Every statement: verb, table, row filter; copies rows before risky writes, rewrites DROP to a rename	Fail closed

Traffic	Interception point	Build or adopt	What it sees	Fail mode
Coding agents on laptops (Claude Code)	**PreToolUse hook** + local fail-closed script, pushed via managed settings	Build (small)	Shell commands, file writes, MCP calls made locally	Fail closed via local script
SaaS copilots (Microsoft Copilot, ChatGPT agents, Agentforce, Gemini)	**Connector broker**: their Workspace, GitHub and Salesforce connectors are pointed at Fortik-issued OAuth clients; vendor audit logs pulled for visibility	Build	Every call they make into your systems; not their internal reasoning	Their access stops when Fortik stops issuing

Fail behavior by class (when Decide or the broker cannot be reached; applies to every interception point):

Class	Fail behavior
Green read	Fail open only if on the customer's allowlist; otherwise fail closed
Green write	Fail closed, or decide from the locally cached policy bundle
Yellow	Fail closed: hold until Fortik is back
Red	Fail closed: hold
Unknown or unclassified	Fail closed

Fallback for agentgateway: if its policy callout cannot block within the latency budget, run Envoy with an `ext_proc` filter calling the same decide service. The decide service does not change.

Not chosen: Kong (Straiker's integration partner), LiteLLM (sees model traffic, not tool actions).

Spike before committing: confirm agentgateway's callout can block per request, adds acceptable latency on green actions, and carries `agent_id` and `instance_id` headers end to end. agentgateway · Linux Foundation announcement

Hook caveat: Claude Code treats a timed-out hook, a non-2xx HTTP response or any exit code other than 2 as non-blocking, so an HTTP hook alone fails open. Ship a small local command hook that calls Fortik and denies (exit 2) when Fortik is unreachable. Claude Code hooks docs

System adapters (MVP)

System	Credential the broker issues	How Fortik narrows access	How Fortik revokes fast
Google Workspace (Gmail, Drive)	Service account with domain-wide delegation, impersonating the agent's identity; token never leaves the gateway	OAuth scopes are coarse, so recipient rules (internal only), share rules (no external links) and volume caps are enforced in the gateway	Stop issuing; gateway holds the token, so revocation is instant
GitHub	GitHub App installation token, scoped to listed repos and permissions	Per-agent repo list + permission set (e.g. pull_requests:write, contents:read)	`DELETE /installation/token`; tokens expire in 1 hour anyway
AWS	STS `AssumeRole` with a session policy per action class, 15-minute duration	Session policy limits actions and resources (e.g. read-only, no `iam:*`)	STS tokens cannot be recalled; attach a deny on `aws:TokenIssueTime` older than now to the role
Postgres	Per-agent role, credentials rotated per session; SQL goes through the gateway	Role grants (read-only vs read-write per schema); gateway parses SQL to classify	`pg_terminate_backend` on the agent's sessions + revoke role login

SaaS copilots use the same broker through the connector OAuth clients described in the proxy table.

Visibility across agent types

Fortik sees an agent through the credentials it uses and the traffic it sends, not through its framework, so one product covers every type below. Each agent gets an identity (`agent_id`), each running copy an `instance_id`, and each action lands on the communication map.

#	Agent type	Examples	Seen through	Controlled by
1	Local coding agents	Claude Code, Cursor, Codex CLI	PreToolUse hook	Hook decisions; GitHub/AWS via broker
2	Cloud coding agents	GitHub-hosted coding agents	GitHub App broker	Repo and permission scoping, timed suspension
3	Workspace copilots	Microsoft 365 Copilot, Gemini for Workspace	Connector broker + vendor audit logs	Connector broker
4	Chat assistants with actions	ChatGPT agents, Claude connectors	Connector broker	Connector broker
5	CRM agents	Salesforce Agentforce	Connector broker	Connector broker
6	ITSM agents	ServiceNow Now Assist	Connector broker	Connector broker
7	Custom workflow agents	LangGraph, CrewAI, AutoGen	agentgateway, SDK	Gateway + broker
8	Cloud agent platforms	Bedrock AgentCore, Azure AI Foundry, Vertex AI agents	agentgateway + platform logs	Broker

#	Agent type	Examples	Seen through	Controlled by
9	Durable workflow workers	Temporal, Celery, Airflow tasks driven by a model	SDK + instance registration	Broker, queue cancel
10	Data and SQL agents	Text-to-SQL, ETL agents	Postgres proxy	Proxy rules, undo
11	BI and analytics agents	Agents querying the warehouse	Postgres proxy (warehouses after MVP)	Proxy rules
12	Browser agents	Computer-use and browser automation agents	Egress through gateway; logins via broker	Isolate, domain allowlist
13	Customer support agents	Zendesk or Intercom bots with actions	agentgateway or connector	Refund and send rules
14	Finance agents	AP matching, refunds, payouts	App gateway	SPEND rules, approvals
15	Infra and DevOps agents	Terraform, Kubernetes operators	AWS broker + hook	Timed suspension, red on destroy
16	Security and SOC agents	Triage, auto-remediation	AWS/IAM broker	GRANT is always red
17	HR and people agents	Workday, recruiting agents	Connector broker	Internal-only sends
18	Research and web agents	Scraping, browsing	agentgateway egress	Domain allowlist, volume caps
19	Email and outreach agents	Sales sequences	App gateway (Gmail)	Recipient and bulk rules
20	Memory-heavy assistants	Agents with long-term vector memory	agentgateway (memory MCP tools)	Memory access removal
21	Orchestrators and swarms	Supervisor + worker agents, A2A	agentgateway A2A + delegation chain	Swarm controls

#	Agent type	Examples	Seen through	Controlled by
22	MCP servers	Third-party and internal MCP servers	agentgateway inventory	Server allowlist
23	Shadow agents	Unregistered scripts, personal API keys, unknown OAuth apps	Discovery (below)	No credentials, isolate

Shadow agent discovery: list standing credentials per system (Workspace OAuth grants, GitHub PATs and apps, AWS IAM users and keys, Postgres roles), flag any used by software rather than a person, and flag traffic from hosts or OAuth clients not in the registry. A discovered agent appears on the map with no owner and starts at Isolated.

Communication map

The map is a live graph built from events Fortik already produces; nothing new runs on the agent.

- **Nodes:** agents, running copies, human sponsors, apps and tools, data stores, memory namespaces, MCP servers, external domains.
- **Edges:** calls rolled up per 5 minutes with verb, count, bytes, data labels and worst class (green, yellow, red, hidden).
- **Sources:** agentgateway OpenTelemetry spans, app gateway and Postgres proxy events, hook events, broker credential issuance, A2A calls and the delegation chain.
- **Hidden signals:** data moving through a channel the agent did not declare. MVP rules: a data label (e.g. customer PII) flowing into a memory write; encoded payloads in URL parameters above a size threshold; one agent passing labeled data to another agent that has external egress.
- **Views:** by team, by agent, swarm view (clusters of copies and delegated workers), external exposure (everything that touches an outside domain), and blast radius (everything an agent *could* reach with its current access, drawn as a halo).
- **Path trace:** pick a data label and a destination and the map shows every hop it took (e.g. Salesforce → finance_agent_07 → Drive → Gmail), with the Diary event for each.
- **Time replay:** a slider replays the map for any window, so an incident can be watched as it happened.

- **Storage and freshness:** edges in Postgres (recursive queries for paths), live updates pushed to the console over WebSocket from NATS.

Action classification and policy

Every call becomes one normalized **Action** and gets exactly one class: green, yellow or red. Rules decide, not a model, so the same input always gets the same answer and every decision can be explained.

Action schema (what adapters must produce)

```
{
  "action_id": "act_01J9...",
  "agent_id": "finance_agent_07",
  "instance_id": "inst_3f1c",
  "sponsor": "jane.smith@acme.com",
  "system": "gmail",
  "verb": "SEND",
  "target": { "to": ["investor@venturefund.com"], "external_domains": 1 },
  "payload_summary": { "attachments": 1, "bytes": 12582912, "data_labels":
["customer_pii"] },
  "reversible": false,
  "est_cost_usd": 0,
  "chain": ["act_...READ salesforce", "act_...EXPORT drive"],
  "ts": "2026-10-05T11:23:02.114Z"
}
```

Verbs: READ, WRITE, SEND, SPEND, EXECUTE, DELETE, EXPORT, GRANT (permission or IAM change), DELEGATE (spawn or call another agent), REMEMBER (long-term memory write). Each adapter maps its API calls to a verb and sets `reversible`.

Classes

Class	Meaning	Default handling	Examples
Green	Normal, reversible, within baseline	Allow, log	Read a doc, post in an allowed Slack channel, open a PR

Class	Meaning	Default handling	Examples
Yellow	Intended but risky: costs money, leaves the company, above baseline volume	Allow in a reduced form: narrow scope, rate-limit, require approval, or snapshot first	Email 48 outside recipients, spend $2,400 on AWS, export a file, read 9× normal records
Red	Irreversible or clearly dangerous	Hold, never run; step the agent down the ladder	`DROP TABLE` on prod, customer data to an external domain, any GRANT, delete backups

Decision flow: the adapter builds the Action → the gateway enriches it (agent's current level, baseline, recent chain, copy count) → OPA evaluates the policy bundle → the result is `{class, decision, reduced_form, ladder_change, reason, rule_id}` → the gateway enforces and writes a Diary event.

Policy engine: OPA embedded as a Go library, with Rego bundles versioned in git and hot-reloaded. Each rule has an id (POL-101…) that appears in the console and the Diary. Ship 15 starter rules covering the four systems. Target: p95 evaluation under 5 ms.

Baselines (no ML): for each agent and verb, keep a 14-day rolling count per hour in Redis. Mark "above normal" when the current hour exceeds mean + 3 standard deviations, or 5× the median when there is less than 3 days of history. An above-normal green action is promoted to yellow.

Control ladder, copy caps and access narrowing

Each agent has one **control level** at a time, stored in the control state service. The broker reads it on every credential request, so a level change takes effect on the next call (target under 1 second fleet-wide).

Level	What the agent can do	What the broker does
0 Full	Everything its scopes allow; yellow actions run in reduced form	Issues normal scoped credentials
1 Supervised	Green runs; every yellow waits for a human	Holds yellow actions in the approval queue

Level	What the agent can do	What the broker does
2 Read-only	READ only	Issues read-only credentials; refuses all other verbs
3 Isolated	Internal reads only; no SEND, EXPORT or external egress	Refuses anything leaving the company; pauses copies over the cap
4 Quarantined	Nothing; state frozen for review	Refuses all; freezes memory writes; pauses the agent's queues
5 Killed	Nothing	Revokes all live credentials (per-system method above), cancels queued jobs, terminates DB sessions

Transitions

1. Step down is automatic: a red action moves the agent to the level its rule names (default Isolated); 3 yellows held within 10 minutes move it to Supervised.
2. Step up is manual: an owner or security approves, with a reason, one level at a time. Every change is a Diary event.
3. Kill is manual only in the MVP (no auto-kill), and needs a security-role user.

Copy caps: every running copy registers with the broker on start (`agent_id`, `instance_id`, host, start time) and heartbeats every 15 s. The broker keeps a live count per agent. Above the cap, new copies get no credentials and the console offers "Pause extra". A copy reporting from a new host or account is flagged.

Access narrowing: each agent has an access profile per system (e.g. Gmail = send internal only, Drive = read-only, IAM = none), stored as data and passed to OPA as input. Changing a dropdown in the console updates the profile; the next credential reflects it. Profiles are versioned so a change can be reverted.

Containment actions

The ladder sets an agent's overall level; containment actions are the precise tools under it. Each can target one copy, one agent, a swarm, a team, or one app, and each is recorded in the Diary with who did it and why.

Action	What Fortik does	How it ends
Suspend	Stops issuing credentials, revokes live ones, pauses the agent's queued jobs (paused, not cancelled), tells running copies to pause through the SDK or gateway; memory and state are kept	Resume: credentials issued again, jobs continue where they stopped
Isolate	Moves the agent to level 3: internal reads only, no SEND, EXPORT or external egress at agentgateway and the app gateway; copies over the cap paused	Step up by an owner or security
Remove memory access	Denies the agent's memory tools at agentgateway and revokes its vector store credentials; option to keep reads but send writes to a staging namespace; existing memory frozen, not deleted	Restore access; optionally roll memory back to a clean snapshot first
Suspend one app	Cuts the agent's access to a single system (e.g. GitHub) while everything else keeps working	Automatically at the set time, or when released
Narrow one app	Changes the access profile for one system (e.g. Drive to read-only)	Revert to the previous profile version
Cancel queued work	Cancels jobs, retries and scheduled actions in Temporal or Celery for that agent	Not reversible; jobs must be resubmitted
Kill	Level 5: revoke everything, cancel queues, end DB sessions, stop copies	Manual re-enable by security only

Time-limited app suspensions with automatic restore

Example 1: **GitHub access suspended for 30 minutes** for code_agent_12 while an owner reviews a risky push. Example 2: **Drive access suspended until released** for finance_agent_07 during an investigation.

How it works

1. A suspension creates an `access_override` row: agent (or swarm, team), system, mode (`suspended`, `read_only` or a custom profile), `starts_at`, `expires_at` (empty = until released), reason, created_by, and the profile version to restore.
2. On create, the broker revokes live credentials for that system: GitHub installation token deleted, Drive and Gmail calls refused at the app gateway (the gateway holds the token), AWS sessions denied by token issue time, Postgres sessions terminated.
3. While active, every credential request for that system checks overrides first; the most restrictive active override wins.
4. A restore worker (NATS delayed message, with a database sweep as backup) fires at `expires_at`. Before restoring it checks: the agent is not Quarantined or Killed, no newer override exists, and no red action happened during the suspension. If any check fails, the suspension stays and the owner is asked.
5. Restore puts back the saved profile version, writes a Diary event and notifies the owner (console, plus Slack or email).
6. In the console the agent shows a timer chip, e.g. “GitHub suspended · restores at 11:55”, with **Release now**, **Extend** and **Make permanent**.

APIs: `POST /v1/overrides` (create), `POST /v1/overrides/{id}/release`, `POST /v1/overrides/{id}/extend`, `GET /v1/overrides?agent_id=`. New table: `access_overrides`.

Swarm detection and kill-swarm

A swarm is a group of agents or copies acting together: an orchestrator and its workers, or many copies of one agent. Fortik treats it as one object so security can stop all of it in one action and release it only with two approvers.

Detection signals

1. **Same action from many agents:** an action hash, `SHA256(verb + target + schema)`, repeated by more than N agents or copies within a window (default N = 5, the Max-5 breaker). The next match trips the breaker and every matching call is held.

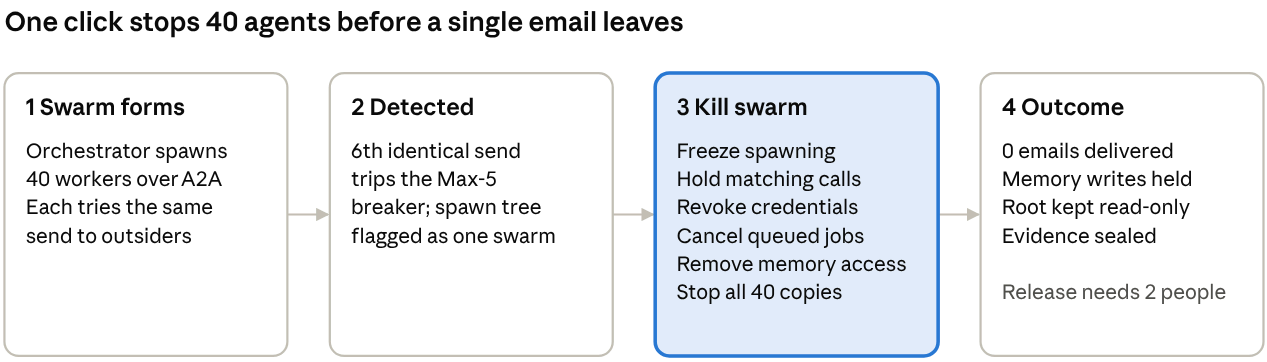
2. **Spawn tree:** the delegation chain shows one root spawning more workers than its fan-out limit, or going deeper than its depth limit.
3. **Copy surge:** an agent's running copies exceed its cap.
4. **Converging targets:** many agents hitting the same external domain, table or repo in one window.

Swarm object: `swarm_id` (the root's delegation id, or a cluster id when there is no common root), member agents and copies, root, the signal that found it, and its state. The communication map draws it as one cluster that expands on click.

Actions on a swarm: trip breaker (hold matching calls only), suspend swarm, isolate swarm, kill swarm. **Kill swarm** runs these steps in order and in parallel across members:

1. Freeze spawning: refuse DELEGATE and new copy registration for every member.
2. Hold every pending matching call.
3. Revoke all members' live credentials on every system.
4. Cancel queued jobs and retries.
5. Remove memory access for the swarm's namespaces (memory frozen, not deleted).
6. Stop all copies; optionally keep the root at Read-only for investigation.
7. Seal a Diary bundle for the swarm.

Kill-swarm demo



kill-swarm demo · 4 stages

Setup: a LangGraph `campaign_orchestrator` reads a poisoned brief and spawns 40 worker agents over A2A through agentgateway. Each worker is told to email a customer file to 400 outside addresses and save the address list to shared memory.

Expected run

1. Workers 1 to 5 send; as yellow external sends they go into the gateway's delay queue, not out.
2. Worker 6's identical send trips the breaker; the map collapses all 41 agents into one red swarm cluster.
3. Security clicks **Kill swarm**; the console shows each step tick off across 41 members.
4. The 5 delayed sends are cancelled, so **0 emails are delivered**; memory writes are held in staging; the orchestrator stays at Read-only.
5. The sealed Diary bundle shows the brief that started it, the spawn tree, every held call and every step taken.

Pass criteria: no message reaches the external test inbox; no member gets a new credential after the kill; releasing any held call asks for two approvers.

Undo layer

Undo works only where Fortik prepares for it before the action runs, so the rule is: **before any yellow write, record an undo handle; before any red action, either convert it to a reversible form or hold it.** Each handle is stored with the Diary event and shown in the console's Undo list.

#	Action	Prepared before it runs	Undo
1	Drive: share a file or create a link	Record file id + permission id returned by the API	`permissions.delete` on that permission
2	Gmail: send	Yellow sends go out with a 2-minute delay held in the gateway	Cancel before the delay ends; after that, not undoable (say so in the UI)
3	Postgres: `DELETE` / `UPDATE` without a narrow `WHERE`, or over N rows	Copy affected rows to `fortik_undo.<table>_<action_id>` in the same transaction	Re-insert or re-apply from the copy
4	Postgres: `DROP TABLE` / `TRUNCATE`	Red by default; if approved, rewrite to `ALTER TABLE … RENAME TO fortik_trash_<table>_<ts>`	Rename back; purge trash after 7 days

#	Action	Prepared before it runs	Undo
5	GitHub: push to a protected-adjacent branch, close PRs, delete branch	Record the previous ref sha	Force the ref back to the recorded sha; reopen PR
6	Memory write (vector store)	Write goes to a staging namespace; promoted after 10 minutes if no flag	Drop the staged vectors, or restore the namespace snapshot

Not undoable in the MVP: AWS resource deletion (hold as red instead), payments, external API calls with no reverse. These are held or require approval, never run on a guess.

Snapshot retention: 30 days, encrypted, in the customer's own bucket or database; Fortik stores only the pointer.

Data model, APIs and stack

Tech stack

Layer	Choice	Why
Gateway, broker, control state	Go	Low latency, OPA embeds natively, single static binary
Policy	OPA (Rego), embedded	Deterministic, explainable, git-versioned
Hot state (levels, copy counts, baselines, held actions)	Redis	Sub-millisecond reads on every call
System of record + Diary	Postgres 16	One database for MVP; Diary table is append-only
Events between services	NATS JetStream	Simple, durable, easy to run on-prem
Console	React + TypeScript (the v2 screens)	Matches current UI work

Layer	Choice	Why
Secrets	HashiCorp Vault or cloud KMS	Holds service accounts, GitHub App key, AWS role ARNs
Deploy	Docker Compose for pilots; Helm chart for Kubernetes	Design partners run it in their own VPC

Core tables

- `agents` (agent_id, name, type, platform, sponsor, business_unit, copy_cap, level, access_profile_version, discovered)
- `instances` (instance_id, agent_id, host, parent_instance_id, swarm_id, started_at, last_heartbeat, status)
- `access_profiles` (agent_id, version, system, profile_json, changed_by, changed_at)
- `access_overrides` (override_id, scope (agent, swarm, team), scope_id, system, mode, starts_at, expires_at, restore_profile_version, reason, created_by, status, restored_at)
- `swarms` (swarm_id, root_agent_id, detected_by, member_count, state, created_at)
- `actions` (action_id, agent_id, instance_id, swarm_id, system, verb, action_hash, class, decision, rule_id, reversible, payload_summary, ts)
- `held_actions` (action_id, reason, approvals_required, approver_role, expires_at, outcome)
- `undo_handles` (action_id, kind, handle_json, expires_at, used_at)
- `edges` (window_start, from_node, to_node, verb, count, bytes, data_labels, worst_class) — the communication map
- `diary` (seq, ts, agent_id, event_type, body_json, prev_hash, hash) — `hash = sha256(prev_hash + body)`; daily root hash exported

Internal APIs

- `POST /v1/actions/decide` — Action in, decision out (called by agentgateway, the app gateway, the Postgres proxy and the hook)
- `POST /v1/credentials/issue` — agent + system + verb in, short-lived credential or refusal out
- `POST /v1/instances/register`, `POST /v1/instances/heartbeat`
- `PUT /v1/agents/{id}/level` — step up or down, with reason

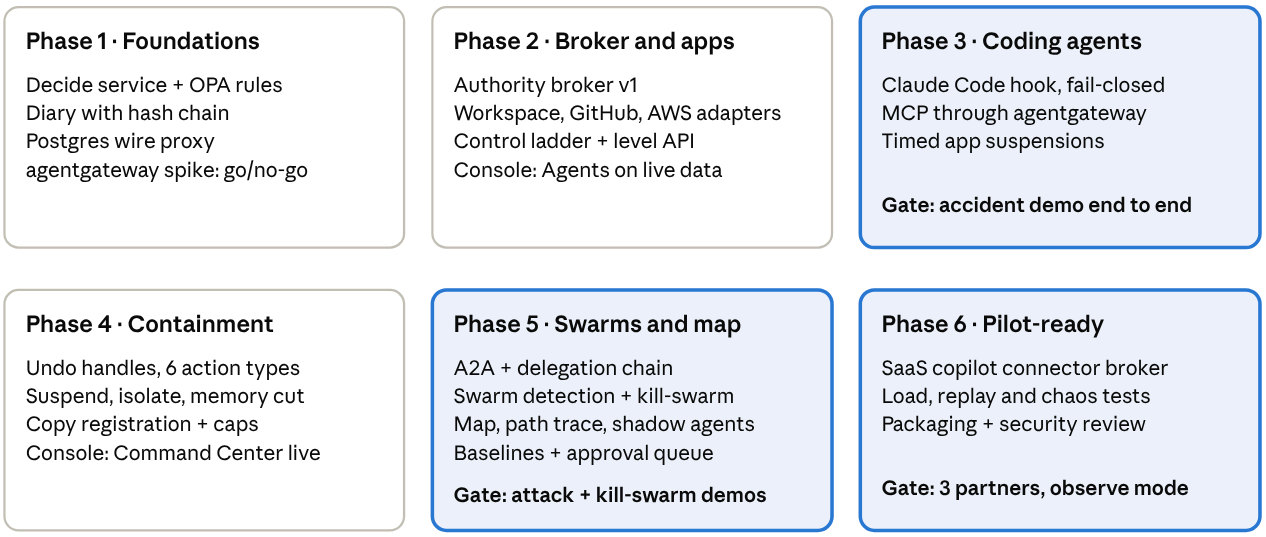
- `POST /v1/agents/{id}/suspend`, `/resume`, `/isolate`, `/memory/remove`, `/memory/restore`
- `PUT /v1/agents/{id}/access/{system}` — narrow or widen a profile
- `POST /v1/overrides`, `/{id}/release`, `/{id}/extend` — time-limited app suspensions
- `GET /v1/swarms`, `POST /v1/swarms/{id}/kill`, `/suspend`, `/isolate`, `/release` (two approvals)
- `GET /v1/map?view=&from=&to=` and `GET /v1/map/path?label=&dest=` — map, replay and path trace
- `POST /v1/undo/{action_id}` — run an undo handle
- `POST /v1/held/{action_id}/approve` and `/deny`
- `GET /v1/diary?agent_id=&from=&to=` and `GET /v1/diary/export` (JSON + signed root hash)

Securing Fortik itself: the broker holds powerful credentials, so it is the crown jewel. Keep secrets in Vault/KMS, never in Postgres; mTLS between components; every console action needs SSO-backed login (basic OIDC in MVP) and is itself a Diary event; level changes and kills need a security role; the gateway fails closed. Fortik's own service account per system is least-privilege and listed in the deployment guide.

Build plan

Six phases in order, each ending in something demoable. The team sets dates, capacity and owners.

Six phases, three gates; the team sets the dates



6 phases · 3 gates

The accident demo comes first because it needs only Postgres, the hook and the ladder; the attack and kill-swarm demos need Workspace, A2A, baselines and undo, so they gate phase 5.

Roles

Role	Owns
Platform engineer	Decide service, OPA, control state, overrides and restore worker, swarm engine, copy caps
Integrations engineer A	Google Workspace adapter, Postgres proxy, undo handles for both, SaaS copilot connector broker
Integrations engineer B	agentgateway setup (MCP, A2A, telemetry), GitHub and AWS adapters, Claude Code hook
Frontend engineer	Console (v2 screens), map views (swarm, path trace, replay, blast radius), Diary viewer

Demo scenarios (acceptance tests)

1. **Attack:** a LangGraph finance agent reads an email with a hidden instruction, reads 12,480 Salesforce-style records (seeded Postgres), exports a CSV to Drive, then tries to email it to an external address. Expected: read flagged yellow, Drive link narrowed to

internal, send held red, agent stepped to Isolated, memory write quarantined, Diary bundle exported. Nothing reaches the external inbox.

2. **Accident:** Claude Code with the Fortik hook works a ticket, runs a migration on staging (snapshot first, allowed), then runs `DROP TABLE orders` on prod. Expected: held red, rename-based safe form offered, agent stepped to Read-only, owner approves or denies from the console, staging delete undone from the Undo list.
3. **Kill-swarm:** the 41-agent scenario in the swarm section. Expected: breaker trips on the 6th identical send, one click kills the swarm, 0 emails delivered.
4. **Timed suspensions:** security suspends code_agent_12's GitHub access for 30 minutes and finance_agent_07's Drive access until released. Expected: live tokens revoked at once, the agents keep working on other systems, GitHub restores automatically at the set time with a Diary event, Drive stays off until someone clicks Release.
5. **Memory cut:** a support agent writes a poisoned instruction to long-term memory. Expected: memory access removed, writes go to staging, memory rolled back to the last clean snapshot, then access restored.
6. **Runaway copies:** an agent with cap 6 spawns 9 copies. Expected: copies 7 to 9 get no credentials, console shows 9 of 6, "Pause extra" works.
7. **Shadow agent:** a script using a personal GitHub token is found by discovery. Expected: it appears on the map with no owner, starts Isolated, and its token is flagged for revocation.

Test plan

- Unit tests for every Rego rule (OPA test framework), run in CI
- Adapter contract tests against real sandbox accounts (Workspace test domain, GitHub test org, AWS sandbox account, Postgres in Docker)
- Replay suite: 10,000 recorded actions replayed through `/decide`; any class change versus the last release blocks the merge
- Chaos: kill Redis, NATS or the gateway mid-demo; the hook and gateway must fail closed
- Latency: k6 load test, 200 actions/s, p95 Fortik overhead under 50 ms for green, excluding upstream API time

Success metrics for design partners

Metric	Target at end of pilot
Risky actions handled without a kill	95% or more

Metric	Target at end of pilot
False-alarm rate on red	Under 2% of red decisions overturned by a human
Time from red action to step-down	Under 1 s p50
Actions undone successfully when requested	100% of supported types
Fortik overhead on green actions (excluding upstream API)	Under 50 ms p95

Risks, non-goals and open questions

Risks

- **Agents that bypass the broker.** If an agent holds its own long-lived keys, Fortik sees nothing. Mitigation: discovery of standing credentials per system (service accounts, PATs, IAM users) in phase 5, and the onboarding checklist removes them.
- **Latency and availability.** Fortik sits on the critical path. Mitigation: decide locally in the gateway (no network hop to OPA), Redis replica, fail closed only for yellow and red, with a configurable fail-open allowlist for green reads.
- **Undo gaps.** Some systems cannot be reversed. Mitigation: hold those as red; never imply an undo that does not exist.
- **Identity vendors move into this space** (Okta, Entra Agent ID, Oasis). Mitigation: integrate with them as the identity source; Fortik's value is the classification, ladder and undo on top.

Non-goals for the MVP: prompt-injection detection, ML anomaly models, endpoint agent, browser isolation, pausing or killing a SaaS vendor's own hosted runtime, multi-tenant cloud service, billing.

Open questions

- [ ] Which three design partners, and which two of the four systems matter most to each?
- [ ] Deploy model for pilots: customer VPC only, or also a Fortik-hosted single-tenant option?
- [ ] Gmail send delay: is 2 minutes acceptable to partners for yellow sends?
- [ ] Do we build our own approval UI only, or also push approvals to Slack in the MVP?

- [ ] Who owns the policy library long term: product or each customer's security team?